Two Pointer Technique (Arrays)

The **Two Pointer Technique** is a method where we use **two indices (pointers)** to traverse an array instead of just one. It helps reduce time complexity and is mostly used in **sorted arrays**.

Instead of checking every pair using nested loops ($O(n^2)$), we use two pointers to solve problems in **O(n)** time.

How it works

- Place one pointer at the **start (left)** and one at the **end (right)**
- Move the pointers based on a condition
- Continue until they meet

When to use

- Array is **sorted**
- Finding **pairs or triplets**
- Problems involving **sum or comparison**

Example Problem

Find if there exists a pair with sum = 10 in array:
`[1, 2, 3, 4, 6]`

Approach

- left = 0 (value = 1)
- right = 4 (value = 6)

Dry Run

1. **Step 1:**
   1 + 6 = 7 → less than 10 → move left → left = 1
2. **Step 2:**
   2 + 6 = 8 → less than 10 → move left → left = 2
3. **Step 3:**
   3 + 6 = 9 → less than 10 → move left → left = 3
4. **Step 4:**
   4 + 6 = 10 → **Found the pair**

Key Idea

- If sum is **small** → **move left pointer forward**
- If sum is **large** → **move right pointer backward**

Code (Java)

```
int left = 0, right = arr.length - 1;

while (left < right) {
    int sum = arr[left] + arr[right];

    if (sum == target) {
        System.out.println("Pair found");
        break;
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
```

Prefix Sum (Cumulative Sum) — Simple Notes

The **Prefix Sum technique** is used to quickly calculate the **sum of any subarray**. Instead of adding elements again and again, we precompute sums and reuse them.

Basic Idea

We create a new array called **prefix array**, where:

- `prefix[i] = sum of elements from index 0 to i`

Formula

- prefix[0] = arr[0]
- prefix[i] = prefix[i-1] + arr[i]

To find sum from index **L to R**:

- **sum = prefix[R] - prefix[L-1]**
- If L = 0 → sum = prefix[R]

Example

Array: `[2, 4, 1, 3, 6]`

Prefix array:

- prefix[0] = 2
- prefix[1] = 2 + 4 = 6
- prefix[2] = 6 + 1 = 7
- prefix[3] = 7 + 3 = 10
- prefix[4] = 10 + 6 = 16

Final prefix array: `[2, 6, 7, 10, 16]`

Dry Run (Find sum from index 1 to 3)

We want sum of `[4, 1, 3]`

Using formula:

- sum = prefix[3] - prefix[0]
- sum = 10 - 2 = **8**

Why it is useful

- Without prefix sum → each query takes O(n)
- With prefix sum → each query takes O(1)

Code (Java)

```
int[] prefix = new int[arr.length];

prefix[0] = arr[0];

for (int i = 1; i < arr.length; i++) {
    prefix[i] = prefix[i - 1] + arr[i];
}

// sum from L to R
int sum;
if (L == 0) sum = prefix[R];
else sum = prefix[R] - prefix[L - 1];
```

Sliding Window Technique

The **Sliding Window Technique** is used to efficiently work with **subarrays or continuous elements**. Instead of recalculating every subarray, we maintain a **window (range)** and slide it forward.

Basic Idea

- Maintain a window using two pointers: **start** and **end**
- Expand the window → add new element
- Shrink the window → remove old element

When to use

- Problems involving **subarrays / substrings**
- Fixed size window (size = k)
- Variable size window (based on condition)

Example Problem

Find the **maximum sum of subarray of size k = 3**
Array: `[2, 1, 5, 1, 3, 2]`

Approach

- First window: sum of first k elements
- Then slide the window:
    - Add next element
    - Remove first element of previous window

Dry Run

1. First window (2,1,5) → sum = 8
2. Slide:
    - Remove 2, add 1 → sum = 7
3. Slide:
    - Remove 1, add 3 → sum = 9 ✅ (max)
4. Slide:
    - Remove 5, add 2 → sum = 6

Max sum = **9**

Key Idea

- Don’t recompute sum again and again
- Just **add new element & remove old element**

Code (Java)

```
int k = 3;
int sum = 0;

// first window
for (int i = 0; i < k; i++) {
    sum += arr[i];
}

int maxSum = sum;

// slide window
for (int i = k; i < arr.length; i++) {
    sum += arr[i];       // add next
    sum -= arr[i - k];   // remove previous
    maxSum = Math.max(maxSum, sum);
}
```
